ALMANTIC

Project & Technical Audit

Prepared for Mazharul Islam
Live application: https://almantic.xyz/
Repository: https://github.com/mazhar1419/almantic
Static source and schema review — not a penetration test or production certification

Almantic — Project & Technical Audit

Prepared for: Mazharul Islam

Application: almantic.xyz — https://almantic.xyz/

Public repository: github.com/mazhar1419/almantic — https://github.com/mazhar1419/almantic

Review basis: uploaded application ZIP, uploaded SQL dump, project README/update notes, Laravel routes/controllers/views, and database schema files

Review type: static source and schema review; not a penetration test or a live production certification

1. Executive Summary

Almantic is a multi-workspace business operations application for freelancers, agencies, and small-to-medium businesses. The source reviewed contains a broad set of connected workflows: custom authentication, workspace creation and switching, CRM, deals, project/task management, invoicing, payment records, files, team chat, notifications, calendar activities, reports, member permissions, and account security pages.

This is a strong flagship portfolio project because it demonstrates more than basic CRUD. It combines multiple business domains and requires careful handling of workspace boundaries, user roles, invoice totals, task/project relationships, collaboration, and account lifecycle operations.

However, the uploaded archive also contains sensitive deployment artifacts. **Do not publish the uploaded archive or SQL dump as-is.** The archive includes a non-example `.env` with non-placeholder values, a SQLite database file, SQL exports, and bundled dependencies. Rotate any secrets that may have been exposed, and inspect repository history if any such material was pushed.

The review also found deployment and assurance gaps that should be addressed before presenting the application as fully production-audited: background Web Push delivery is not configured; the scheduler dispatch route deserves protection review; the custom migration runner needs strict operational controls; and end-to-end tests against the target database and deployed environment are not evidenced by the supplied documentation.

Overall assessment

Area	Assessment from inspected source	Confidence
Product breadth	Broad, integrated agency operations feature set	High
Multi-workspace model	Workspaces, memberships, invitations, and permissions are present	High
CRM / project / invoice workflows	Routes, views, controller logic, and schema support these areas	High
Authentication	Custom auth, verification, reset, rate-limit, remember-token flows are present	High
Authorization correctness	Checks exist in controllers; requires systematic cross-tenant testing	Medium
Financial correctness	Invoice items, recalculation, payments, and overpayment handling are present; edge-case tests still needed	Medium
Email reliability	Provider configuration dependent; delivery needs live end-to-end	Medium

	verification	
Background notifications	Not fully implemented for server-originated Web Push while app is closed	High
Production readiness	Not independently certified; remediation and testing remain	High
Secret/data hygiene	Uploaded archive includes sensitive-looking environment and database artifacts	High

This is a source-based assessment. It does not claim that every workflow was executed in a browser against the production database.

2. Product Definition

Target customer

Small and medium agencies, freelancers, consultants, and businesses that need one place to manage:

• Prospect and customer relationships.

• Leads, deals, and follow-ups.

• Client projects and task delivery.

• Invoices and payment records.

• Team membership, permissions, and conversations.

• Files, calendar activities, and business reports.

Main product promise

Connect sales and client delivery in one workspace: capture a lead, develop the relationship, track a deal, organize project work, issue an invoice, record payments, and coordinate the team without scattering the workflow across unrelated tools.

Portfolio value

The project can demonstrate:

1. A multi-tenant/workspace-oriented application model.

2. Authentication and account lifecycle workflows.

3. Server-side role and module/action permission checks.

4. Relational business data modeling.

5. Invoice line items and balance recalculation.

6. Project and task relationships.

7. Collaboration and notification flows.

8. File upload/download authorization.

9. SQL migration management.

10. A realistic combination of product requirements and engineering trade-offs.

3. Review Scope and Method

Reviewed:

• `routes/web.php`.

• Authentication, account, dashboard, workspace, collaboration, resource, utility, and migration controllers.

• Custom authentication middleware and user model.

• Blade views for CRM, projects, tasks, invoices, chat, calendar, reports, settings, files, and account security.

• Composer and frontend package manifests.

• SQL migration files and the supplied database dump.

• Existing README and update notes.

• Environment-file presence and sensitive artifact categories, without reproducing secret values.

Not performed:

• A complete browser-driven acceptance test of every workflow.

• A live production database test suite.

• A penetration test or third-party security scan.

• A review of every dependency for known vulnerabilities.

• A verification of mail deliverability to multiple mailbox providers.

• A load, concurrency, or performance test.

• A legal/compliance audit.

4. Feature Inventory

4.1 Authentication and account lifecycle

Source routes and controllers cover:

• Registration and login.

• Logout.

• Email verification.

• Verification resend.

• Forgot-password and reset-password flow.

• Remembered-login flow.

• Saved-device listing and revocation.

• Login activity.

• Profile editing.

• Email and password change confirmation.

Security-related implementation also includes custom authentication middleware, login-attempt/rate-limit records, email verification records, password reset records, remembered-login tokens, and profile change tokens.

Review note: custom authentication means the application must own all the edge cases normally provided by a mature authentication package. Regression tests should cover session rotation, CSRF, token expiry, token reuse, password reset, email change, account enumeration, rate limits, and device revocation.

4.2 Workspaces and multi-tenancy

The source contains:

• Workspace listing and selection.

• Workspace creation.

• Active workspace resolution.

• Invitation creation and invitation-token handling.

• Membership listing and role changes.

• Member banning/suspension behavior.

• Module/action permission configuration.

• Workspace settings.

• Workspace activity log.

• Invitation cancellation.

The schema includes `workspaces`, `workspace_members`, `workspace_invitations`, and `workspace_member_permissions`.

Review note: many resource queries include a workspace ID, and controllers perform membership/role checks. That is a good foundation. Multi-tenancy should still be tested as a security boundary: a user in workspace A must not read, update, delete, download, or infer records belonging to workspace B by changing a URL ID or form field.

4.3 CRM

The resource controller and views support:

• Customers.

• Companies.

• Contacts.

• Leads.

• Deals.

• Activities.

The schema includes corresponding relational tables. Records support operational fields such as contact details, statuses, notes, values, due dates, and company/customer associations.

Review note: ensure cross-record references are always validated against the current workspace. In particular, contact/company relationships, project/customer relationships, invoice/customer relationships, and task/project relationships should reject IDs belonging to another workspace.

4.4 Projects and tasks

The application includes:

• Project records and details.

• Project status, priority, dates, description, customer association, and budget.

• Task records associated with projects.

• Task status, priority, and due dates.

• Project task board/Kanban view.

• Project member management.

• Project chat and membership controls.

• Project/task summaries in reports.

Tables include `projects`, `tasks`, `project_members`, and `project_comments`.

Review note: test that task records cannot be attached to another workspace's project and that project-specific chat access follows the same membership rules as project access.

4.5 Invoices and payment records

The source supports:

• Invoice creation, viewing, editing, and deletion.

• Invoice line items.

• Invoice numbering within workspace scope.

• Currency, issue date, due date, tax rate, discount, status, and notes.

• Print-friendly invoice output.

• Payment recording against an invoice.

• Invoice balance/status recalculation.

• Overpayment protection.

Tables include `invoices`, `invoice_items`, and `payments`.

Review note: this is invoice/payment record management, not a payment gateway. Before production use, test rounding rules, zero/negative values, duplicate payment submissions, concurrent payments, deletion/editing of invoices with existing payments, currency consistency, tax/discount ordering, and transaction rollback on partial failure. Financial calculations should use a consistent decimal strategy and database precision.

4.6 Chat and collaboration

The source includes:

• Workspace channels.

• Direct chat request/approval flows.

• Conversation participants.

• Message sending and polling.

• Message search.

• Shared notes.

• Project chat.

• Project member management.

• Direct-conversation controls.

• Project chat mute controls.

• Collaboration notifications.

Tables include `chat_messages`, `chat_conversations`, `chat_participants`, `chat_requests`, `chat_general_requests`, `chat_conversation_messages`, `chat_conversation_notes`, `chat_user_controls`, and `project_chat_mutes`.

Review note: chat is implemented with HTTP polling in the reviewed route structure. Do not claim WebSocket-based real-time delivery unless an additional transport exists. Test conversation membership, workspace isolation, message search authorization, note editing permissions, and safe rendering of user-generated message content.

4.7 Calendar, scheduled activities, and notifications

The source includes:

• Calendar route and month-grid view.

• Scheduled activity create/update/delete routes.

• Due notification dispatch logic.

• Notification list and polling.

• Mark-as-read route.

• Browser service-worker notification handlers.

The update notes say scheduled activities can be edited/deleted by the creator, workspace owner, or workspace admin, with controller-side checks.

Important limitation: the supplied notes explicitly state that background Web Push delivery while the app is closed is not configured because VAPID keys, stored push subscriptions, and a Web Push sender are missing. Browser notifications can be triggered while the application is open after permission is granted, but that is not equivalent to a complete push delivery system.

Priority action: review and protect `/system/scheduler/due-notifications`. In the visible route definition, it is not grouped under the authenticated application middleware and has no visible route-level authentication middleware. Confirm the handler is safe to invoke publicly; preferably protect it with a scheduler secret, authenticated internal route, or host-level access control. Ensure it cannot be abused to trigger excessive work or expose notification data.

4.8 Files

The source supports:

• Workspace file upload.

• File metadata storage.

• Authenticated file download.

• File deletion.

• Permission checks for file operations.

• UUID-based stored filenames.

• MIME/extension validation and a 20 MB upload limit in the reviewed utility controller.

Files are stored via Laravel's local filesystem disk in workspace-specific paths.

Review note: verify that the configured disk is private, uploaded files cannot be executed, MIME checks are sufficiently strict, file downloads always require active workspace membership, storage backups exist, and deletion/retention behavior is intentional. Consider malware scanning if untrusted users can upload documents.

4.9 Reports

The reports controller computes project/task and invoice-related summaries and applies role-based restrictions in the visible implementation.

Review note: confirm that all aggregates are workspace-scoped and that member/project visibility rules are consistent between detailed records and summary counts. Reports should not be marketed as accounting-grade or compliance-certified reports.

4.10 Database and migration approach

The reviewed schema includes 41 table definitions across the supplied SQL materials, covering authentication, workspaces, CRM, project delivery, invoices, collaboration, files, notifications, audit records, and login activity.

The application uses numbered SQL migration files and an `almantic_migrations` registry through a custom `/system/migrate` workflow. The migration runner checks a configured migration key before executing migrations.

Review note: SQL migrations are operationally sensitive. Review every migration before production execution, back up the database, run against a staging copy first, and verify migration checksums/versioning and rollback/recovery procedures. The migration page may be public, but the execution action must remain protected by a strong secret and deployment-level controls.

5. Architecture Summary

Application layers

Layer	Observed implementation
Web entry	Laravel routes and middleware
UI	Blade templates, CSS, vanilla JavaScript
Domain/workflow logic	Workspace resource, collaboration, utility, account, and auth controllers
Data access	Laravel Query Builder and SQL migrations
Relational storage	MySQL/MariaDB-oriented schema
Authentication state	Session-based custom authentication
Collaboration	HTTP routes, message polling, database-backed conversations
File storage	Laravel filesystem abstraction, local disk in supplied controller
Schema changes	Numbered SQL files and custom migration registry

Architectural strengths

• One coherent workspace model links many business domains.

• Controllers include explicit membership and permission checks.

• The code uses workspace IDs in many resource lookups.

• Invoice items and payment records are modeled separately.

• File storage paths are separated from original filenames.

• Account and collaboration workflows have dedicated database records.

• Product modules are grouped in a shared resource controller to reduce repeated CRUD scaffolding.

Architectural trade-offs

• A large shared resource controller can become difficult to test and maintain as rules diverge across modules.

• Custom authentication requires more security regression coverage.

• Manual SQL migrations require strict release discipline.

• Query Builder code needs careful explicit relationship validation.

• Polling-based collaboration can increase request volume and does not provide the same delivery semantics as WebSockets.

• A local filesystem requires deployment-specific persistence and backup planning.

6. Security and Data Hygiene Findings

Critical: non-example environment file in the uploaded archive

The uploaded ZIP contains `almantic/main/.env` with non-placeholder values for application, database, and mail configuration categories, including an application key and database/mail credentials.

Actions:

1. Do not commit or redistribute this archive.

2. Rotate any credentials that may have been exposed to a public repository, shared drive, or untrusted recipient.

3. Verify Git history, not only the current working tree.

4. Remove `.env` from public history if it was committed; coordinate history rewrite and rotate secrets first.

5. Keep `.env.example` limited to empty or clearly fake placeholders.

Critical: SQL dump and SQLite file

The ZIP contains SQL exports and a SQLite database file; the separately supplied SQL dump contains user/workspace insert statements. These may include real account or workspace information.

Actions:

• Do not publish production database dumps.

• Inspect the SQLite database and SQL export for personal data, password hashes, email addresses, tokens, business records, and other sensitive content.

• Replace live data with synthetic seed data for demos.

• If real user data was exposed, follow the applicable privacy and incident-response process.

High: migration and scheduler routes

• The migration execution handler checks a configured migration key, which is positive, but a publicly reachable migration interface still deserves strong operational controls.

• The due-notification scheduler route is visibly outside the authenticated route group. Protect it with a secret or internal-only access control and test repeated calls.

• Ensure secrets are not included in route output, logs, error pages, or deployment artifacts.

High: custom auth and session security

Review and test:

• Session ID rotation after login.

• Session invalidation on logout and password changes.

• CSRF protection on all state-changing browser routes.

• Secure, HttpOnly, SameSite cookies and HTTPS-only production behavior.

• Password hashing configuration.

• Rate limiting on login, registration, verification, resend, and password reset.

• Generic responses that avoid account enumeration.

• One-time token expiry, invalidation, and replay prevention.

• Remember-token hashing, rotation, expiry, and device revocation.

High: tenant isolation

Build automated tests that attempt cross-workspace access for every entity and operation:

• View/update/delete by changing numeric IDs.

• Attach a record from another workspace via a form field.

• Download another workspace's file.

• Search another conversation's messages.

• Access project chat without project membership.

• View aggregate report data not visible to the member.

• Change member role or permissions without owner authority.

Medium: uploads

• Keep uploads outside the public web root.

• Enforce maximum size, extension allow-list, MIME/content checks, and randomized storage paths.

• Consider malware scanning and content-disposition safeguards.

• Test unauthorized download and deletion.

• Define backup and retention policies.

Medium: financial edge cases

• Test duplicate form submission and concurrent payment requests.

• Validate quantity, price, tax, discount, currency, and payment amount boundaries.

• Verify that invoice totals and balances remain consistent after every update.

• Use database transactions around multi-table invoice/payment changes.

• Define rules for editing or deleting invoices that already have payments.

• Do not describe payment records as payment processing.

Medium: dependency and deployment hygiene

• Avoid committing `vendor/` unless there is a deliberate reason.

• Keep lockfiles, `.env.example`, and required source files.

• Review dependency vulnerabilities and update strategy.

• Ensure debug mode is off in production.

• Keep logs free of credentials and sensitive payloads.

• Configure database backups, uptime/error monitoring, and recovery tests.

7. Data Model Inventory

The supplied SQL materials include these table groups (exact schema details should be checked against the migration sequence used in the live deployment):

Identity and access

• `users`

• `sessions`

• `cache`

• `password_resets`

• `email_verifications`

• `login_attempts`

• `auth_rate_limits`

• `remember_tokens`

• `profile_change_tokens`

• `login_activity`

Workspaces and governance

- workspaces
- workspace_members
- workspace_invitations
- workspace_member_permissions
- almantic_migrations
- audit_logs

CRM and sales

- customers
- companies
- contacts
- leads
- deals
- activities
- scheduled_activities

Delivery and billing

- projects
- project_members
- project_comments
- tasks
- invoices
- invoice_items
- payments

Files and collaboration

- workspace_files
- chat_messages
- chat_conversations
- chat_participants
- chat_requests
- chat_general_requests
- chat_conversation_messages

- `chat_conversation_notes`
- `chat_user_controls`
- `project_chat_mutes`
- `app_notifications`

The database has enough breadth to support the central product story: workspace-scoped relationships, sales, project delivery, billing, and collaboration.

8. Testing and Verification Plan

P0 — before sharing or continuing deployment

☐ Rotate any exposed application, database, mail, and migration secrets.

☐ Remove `.env`, database dumps, SQLite files, and private uploads from any public repository/history.

☐ Protect the scheduler route and confirm migration execution cannot be triggered without authorization.

☐ Verify production debug mode is disabled.

☐ Create a synthetic demo account and synthetic sample data.

☐ Test registration, email verification, login, logout, password reset, and invitation mail in the deployed environment.

P1 — authorization and data integrity

☐ Add cross-workspace access tests for every module.

☐ Test workspace switching and stale session context.

☐ Test member roles and every permission action (view/create/edit/delete).

☐ Test project membership and project chat access.

☐ Test invoice calculations, payment balance updates, duplicate submissions, and concurrent updates.

☐ Test file upload/download/delete permissions.

☐ Test chat conversation participant boundaries and search authorization.

☐ Test activity edit/delete permissions and scheduler invocation limits.

P2 — reliability and maintainability

☐ Add repeatable staging deployment and migration backup procedure.

☐ Add error monitoring and structured log review.

☐ Add database backup and restore drills.

☐ Add mail delivery monitoring and a documented retry/support flow.

☐ Configure a real background Web Push pipeline only if required by product scope.

☐ Consider splitting the shared resource controller into module-specific services/controllers as the codebase grows.

☐ Add performance checks for reports, search, chat polling, and large workspaces.

☐ Review accessibility, responsive behavior, and browser compatibility.

9. Screenshot and Portfolio Plan

Create `docs/screenshots/` and use current, anonymized application screenshots.

Recommended sequence:

1. `01-dashboard.png` — dashboard and workspace context.
2. `02-workspace-switcher.png` — personal/team/company workspace options.
3. `03-crm-leads.png` — leads and customer management.
4. `04-deals.png` — sales pipeline.
5. `05-project-kanban.png` — project board and task statuses.
6. `06-invoice.png` — invoice line items, tax/discount, balance.
7. `07-team-chat.png` — direct/project chat.
8. `08-calendar.png` — month-grid calendar and scheduled activity.
9. `09-permissions.png` — member permissions settings.
10. `10-reports.png` — operational reporting.

For each screenshot:

• Use synthetic company and customer details.

• Hide personal email addresses and internal IDs.

• Never show API keys, session tokens, invitation tokens, or migration keys.

• Prefer readable desktop screenshots with consistent dimensions.

• Add concise captions explaining the workflow and its business value.

Suggested case-study narrative

Problem: small agencies often scatter leads, billing, project delivery, and team communication across separate tools.

Approach: build a multi-workspace application that links CRM, sales opportunities, projects, tasks, invoices, payment records, and collaboration under one workspace model.

Engineering challenges: workspace isolation, role-aware permissions, invoice balance integrity, custom authentication, private file access, and coordination across multiple business modules.

Trade-offs: custom authentication and SQL migrations provide direct control but increase testing responsibility; polling simplifies collaboration transport but does not replace WebSockets or background push infrastructure.

Outcome: a live, multi-module application that can be explored through a public registration flow, with selected source code available publicly and the complete application source kept private.

10. Public README Recommendations

The public README should:

• Present Almantic as a unified agency operations SaaS, not only a simple CRM.

• Link to the live application and the public repository.

• State clearly that only selected source files are public.

• Include screenshots of real workflows.

• Explain how a reviewer can register and verify email, including checking Spam/Junk.

• Highlight architecture, data model, authorization, and invoice workflow.

• Clearly state that payments are records rather than gateway processing.

• Avoid unsupported claims such as “fully secure,” “production certified,” “real-time WebSocket chat,” “background push notifications,” or “accounting-grade.”

• Avoid including live demo passwords unless the account is explicitly created for public use and can be safely rotated/reset.

11. Final Recommendation

Almantic should be the flagship project in the developer portfolio because it demonstrates a connected business product rather than a small isolated demo. Its strongest story is the combination of workspace architecture, CRM, billing, project execution, permissions, and collaboration.

Before promoting it heavily to international clients, prioritize:

1. Secret and database artifact cleanup.

2. Scheduler/migration route hardening.

3. Cross-workspace authorization tests.

4. Financial edge-case tests.

5. Reliable email verification and invitation testing.

6. Real screenshots and a guided demo journey.

7. A public README that accurately describes the private-source boundary and known limitations.

Conclusion: strong portfolio breadth and practical engineering scope; production assurance remains a separate workstream that requires systematic tests and deployment review.